A FRAGILE BALANCE OF POWER

Seismic Monitoring and Analysis Platform

Sapienza University of Rome
Laboratory of Advanced Programming
01-04-2026

PROBLEM CONTEXT

The hackathon challenge is set in a future scenario marked by geopolitical tension and limited trust, where seismic sensors are deployed worldwide to monitor critical activities.

The goal is designing a system that reliably processes and delivers critical information under the following conditions:

- Geopolitical scenario with **limited data sharing**
- **Seismic sensors** deployed in remote areas
- Continuous **real-time data generation**
- No processing in the neutral command center
- System components may **fail** unpredictably

MAIN REQUIREMENTS

Functional

- Real-time data ingestion from sensors
- Frequency-based event classification
- Persistent storage of detected events
- Monitoring dashboard (live + historical)

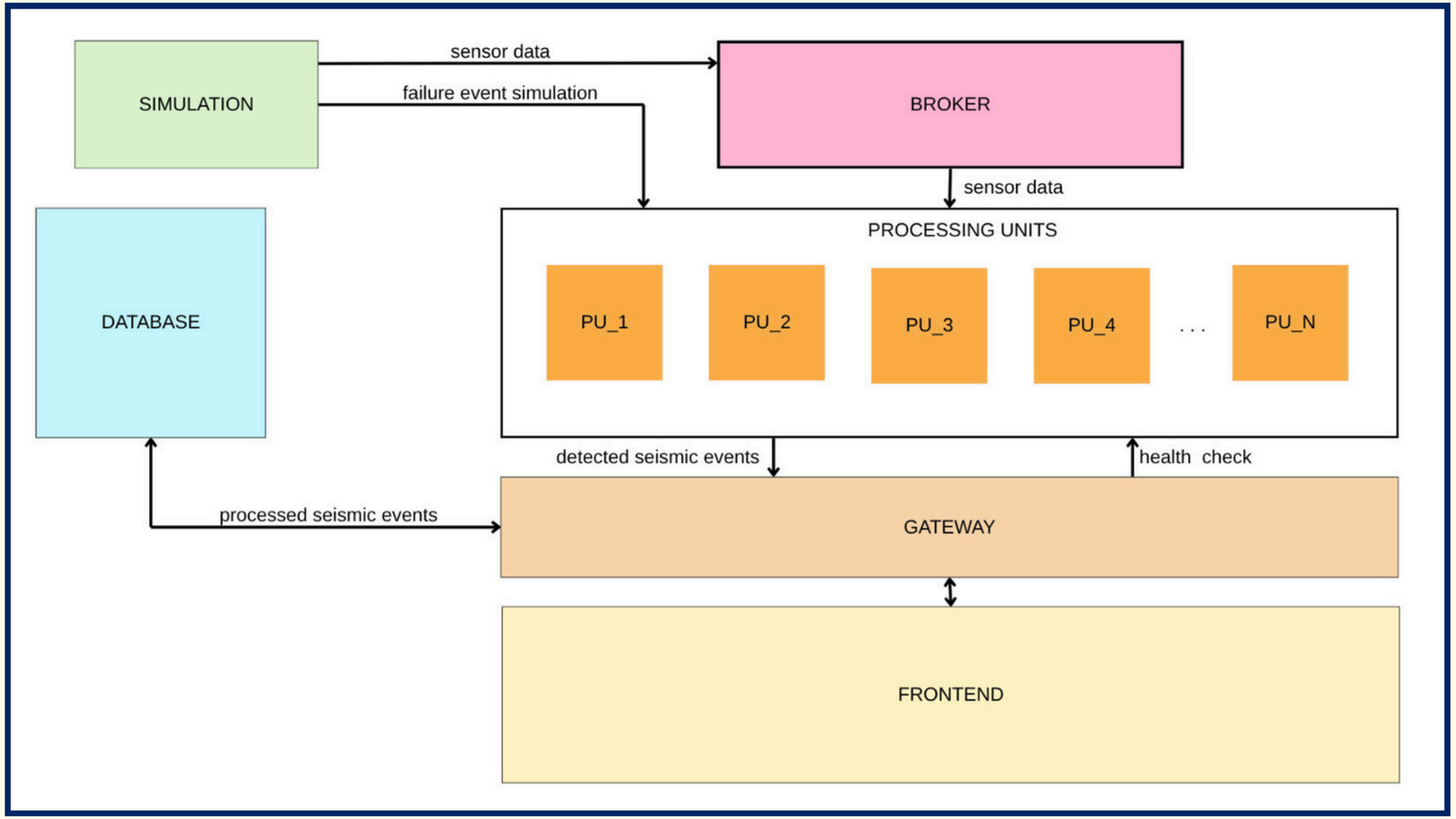
Non-Functional

- Fault tolerance (replicated processing units)
- Distributed architecture (no monolith)
- Duplicate-safe persistence
- Full Docker-based reproducibility

DESIGN STRATEGY

- To address these requirements, we adopted an **event-driven architecture**, where data flows continuously through the system.
- A custom **broker** is used to distribute incoming data to multiple processing replicas, ensuring redundancy.
- The **processing layer** is **replicated** to handle failures and maintain continuous operation.
- A **gateway** component acts as the single entry point, simplifying communication and centralizing persistence and monitoring logic.

ARCHITECTURE OVERVIEW



ARCHITECTURE

- **Simulator** → WebSocket streams (sensor data)
- **Broker** → collects and redistributes data (fan-out)
- **Processing Units** → sliding window, FFT analysis, event classification
- **Gateway** → receives events (HTTP), stores them (deduplicated), monitors replicas
- **Database** → stores data to retrieve history
- **Frontend** → data visualization

Protocols used:

- **WebSocket** → real-time streaming
- **HTTP REST** → inter-service communication
- **SSE** → failure simulation
- **PostgreSQL (asyncpg)**→ database communication

CONCLUSIONS

Results

- Real-time seismic data processing
- Providing a monitoring dashboard
- Correct frequency-based classification
- Fault tolerance through replication
- Event deduplicaton
- Fully reproducible with Docker

Key Takeaways

- Replication + health checks → system resilience
- Event-driven design fits streaming scenarios
- Trade-off between simplicity and scalability

THANK YOU FOR YOUR ATTENTION

AFGS group

Federico Antolini 1985581
Giovanni Biondi 1952718
Andrea Di Vito 1997459
Simone Giudici 1985466